

NAFAD PAY

Architecture AWS — Early Stage (MVP)

Groupe 4 — Analytics & BI · Guide de Déploiement Complet avec Schémas

Région : AWS eu-west-3 (Paris) · Stack : PostgreSQL 16.6 · Metabase v0.52.9 · Docker
Touba Abdellahi · Rakiae Dehah · Khadijetou M. Mewloud · Oum Elvadhli Cheikh · Isshagh Alla

1. Contexte & Vue d'Ensemble

NAFAD PAY est un système fictif de paiement mobile mauritanien. Le Groupe 4 est responsable de la couche Analytics & BI. Ce document couvre le déploiement AWS réel du MVP : création des ressources cloud, configuration réseau, chargement des données et mise en production du dashboard Metabase.

Ressource AWS	Nom	Configuration	Zone
EC2	nafad-metabase-g4	t3.medium · Ubuntu 24.04 · 20 GB	eu-west-3a
RDS	nafad-dwh-g4	db.t3.micro · PostgreSQL 16.6 · 20 GB	eu-west-3c
Security Group	nafad-ec2-sg-g4	Ports 22, 80, 3000 ouverts	VPC défaut
Security Group	nafad-rds-sg-g4	Port 5432 depuis nafad-ec2-sg-g4	VPC défaut
VPC	Default VPC	vpc-032d98cc7bb8d28c4	eu-west-3
Elastic IP	(à allouer)	IP fixe associée à nafad-metabase-g4	eu-west-3a

2. Architecture AWS — Vue Globale

2.1 Schéma d'architecture

Le schéma ci-dessous représente l'infrastructure AWS complète déployée pour NAFAD PAY G4. Les deux composants principaux (EC2 Metabase et RDS PostgreSQL) sont isolés dans deux sous-réseaux distincts au sein du VPC par défaut.

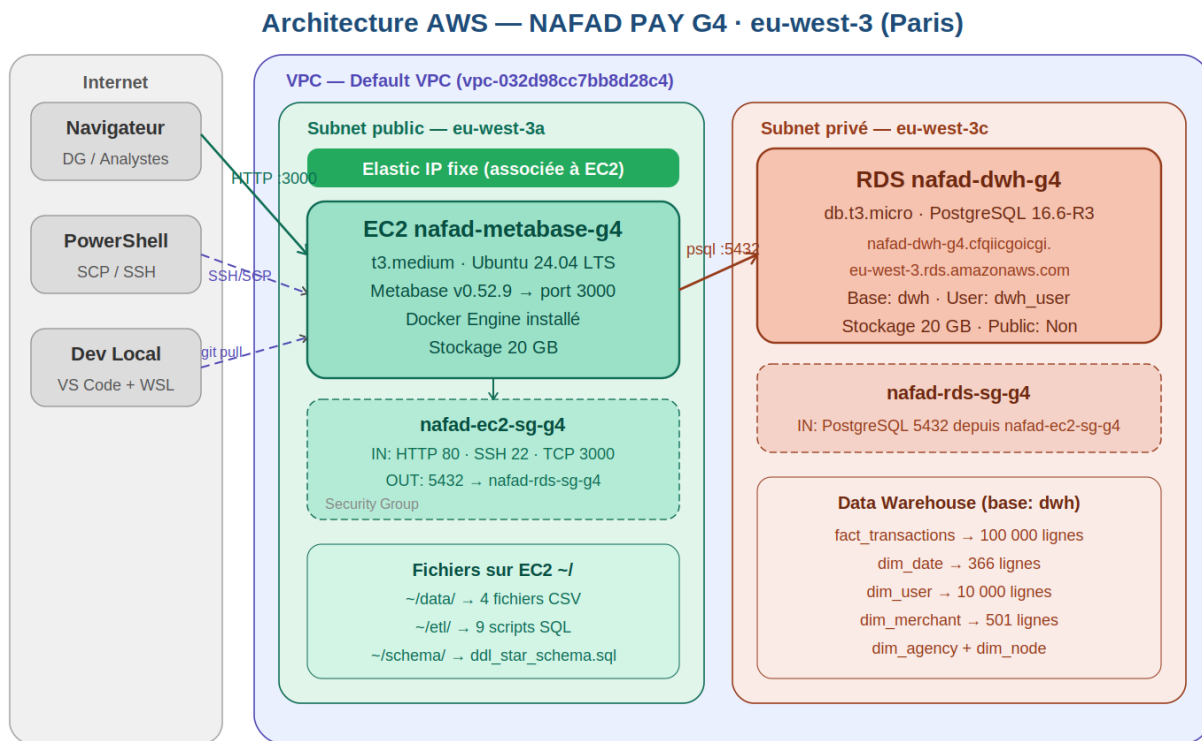


Figure 1 — Architecture AWS complète NAFAD PAY G4 (eu-west-3 Paris)

2.2 Principes d'isolation réseau

L'architecture repose sur deux niveaux d'isolation réseau complémentaires :

- › Subnet public (eu-west-3a) : héberge l'EC2 Metabase — accessible depuis internet sur les ports 80, 22 et 3000.
- › Subnet privé (eu-west-3c) : héberge RDS PostgreSQL — aucun accès direct depuis internet. Accessible uniquement depuis le Security Group de l'EC2.
- › Elastic IP : l'IP publique de l'EC2 est fixée via une Elastic IP pour éviter qu'elle change à chaque redémarrage.

Endpoint RDS (à conserver)

nafad-dwh-g4.cfqiicgoicgi.eu-west-3.rds.amazonaws.com
Base : dwh · Utilisateur : dwh_user · Port : 5432

3. Security Groups — Configuration Réseau

3.1 nafad-ec2-sg-g4 (pour l'EC2 Metabase)

Type	Port	Source	Justification
HTTP	80	0.0.0.0/0	Accès web standard
SSH	22	0.0.0.0/0 (ou My IP)	Administration EC2
Custom TCP	3000	0.0.0.0/0	Interface Metabase dashboard

3.2 nafad-rds-sg-g4 (pour RDS PostgreSQL)

Type	Port	Source	Justification
PostgreSQL	5432	nafad-ec2-sg-g4	Accès exclusif depuis l'EC2 Metabase

Règle de sécurité fondamentale

RDS n'a aucun inbound depuis internet (0.0.0.0/0). La seule source autorisée est le Security Group de l'EC2. Même si quelqu'un connaît l'endpoint RDS, il ne peut pas s'y connecter sans passer par l'EC2.

4. Guide de Déploiement — 11 Étapes

4.1 Flux de déploiement

Le déploiement suit 11 étapes séquentielles, de la création des Security Groups jusqu'à l'association de l'Elastic IP. Le schéma ci-dessous résume ce flux :

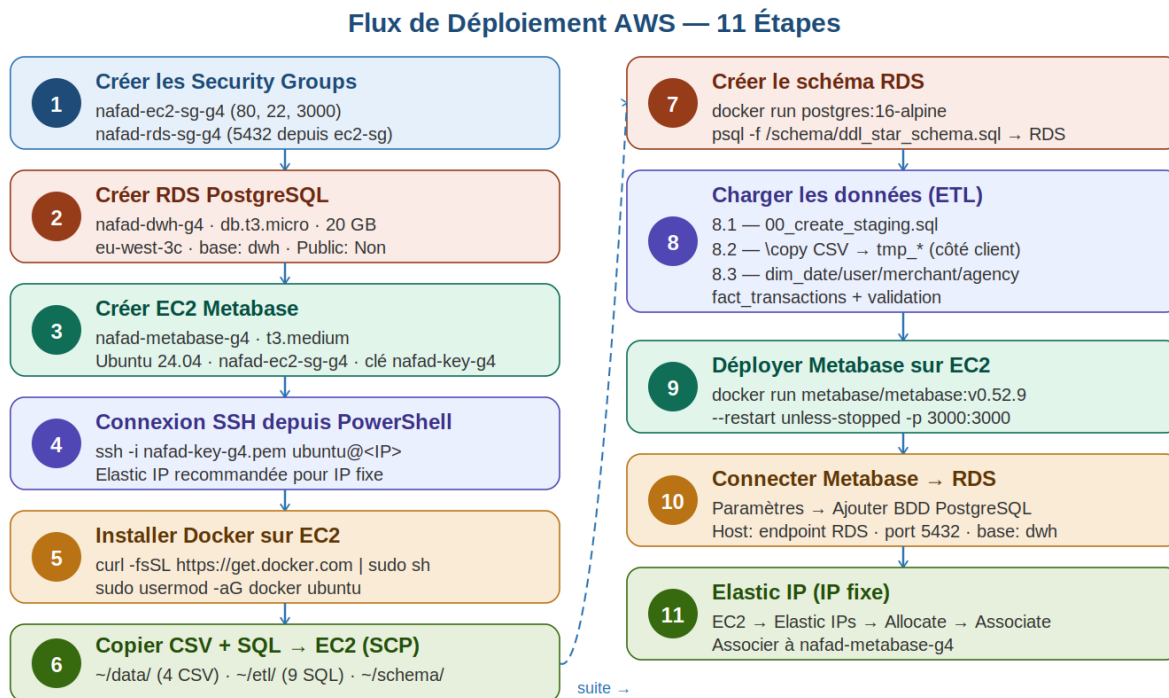


Figure 2 — Flux de déploiement AWS en 11 étapes

4.2 Étapes 1–3 : Infrastructure de base

Étape 1 — Security Groups

Créer les deux Security Groups depuis VPC → Security Groups → Create security group. Créer d'abord nafad-ec2-sg-g4, puis nafad-rds-sg-g4 en référençant le premier comme source.

Étape 2 — RDS PostgreSQL

Créer l'instance RDS depuis RDS → Create database. Sélectionner Free Tier, PostgreSQL 16.6-R3, instance db.t3.micro, Public access: No, Security Group nafad-rds-sg-g4.

Étape 3 — EC2 Metabase

Lancer l'instance EC2 Ubuntu 24.04 LTS t3.medium. Télécharger la clé nafad-key-g4.pem et associer le Security Group nafad-ec2-sg-g4.

4.3 Étapes 4–6 : Configuration de l'EC2

Étape 4 — Connexion SSH (PowerShell Windows)

```
# Fixer les permissions de la clé .pem
icacls "C:\Users\...\nafad-key-g4.pem" /inheritance:r /grant:r "$($env:USERNAME):(R)"
# Connexion
ssh -i "C:\Users\...\nafad-key-g4.pem" ubuntu@<IP_PUBLIQUE_EC2>
```

Étape 5 — Installer Docker sur l'EC2

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker ubuntu
newgrp docker
```

Étape 6 — Copier les fichiers CSV et SQL (SCP depuis PowerShell)

```
# Copier les CSV
scp -i "... \nafad-key-g4.pem" data\stg_users.csv data\stg_merchants.csv `
    data\stg_agencies.csv data\stg_transactions.csv ubuntu@<IP>:~/data/
# Copier les scripts SQL
scp -i "... \nafad-key-g4.pem" -r schema etl ubuntu@<IP>:~/
```

5. Pipeline ETL — Chargement vers RDS

5.1 Schéma ETL détaillé

L'ETL est exécuté via des containers Docker postgres:16-alpine sur l'EC2, qui se connectent à RDS via le réseau VPC privé. Contrainte importante : RDS ne supporte pas COPY depuis un fichier local — la commande \copy (côté client psql) est utilisée.

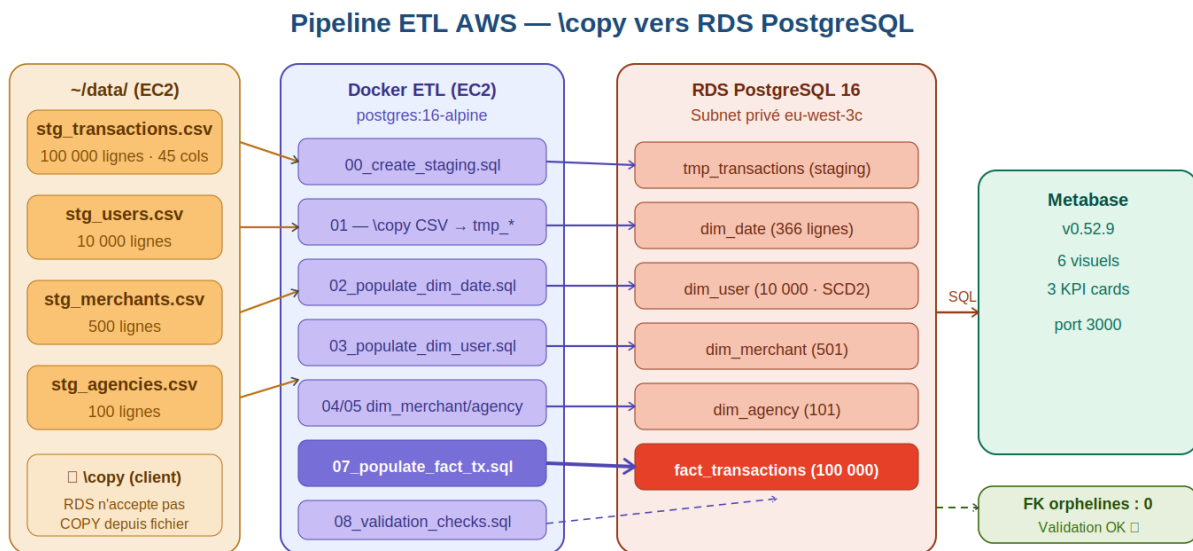


Figure 3 — Pipeline ETL : des fichiers CSV aux tables du Data Warehouse RDS

5.2 Étape 7 — Créer le schéma DDL sur RDS

```

RDS=nafad-dwh-g4.cfqiicgoicgi.eu-west-3.rds.amazonaws.com
docker run --rm \
  -e PGPASSWORD='RGHgv5#Kp9mX2wQ1' \
  -v ~/schema:/schema \
  postgres:16-alpine \
  psql -h $RDS -U dwh_user -d dwh \
  -f /schema/ddl_star_schema.sql
  
```

5.3 Étape 8.1 — Créer les tables staging

```

docker run --rm -e PGPASSWORD='RGHgv5#Kp9mX2wQ1' \
  -v ~/etl:/etl postgres:16-alpine \
  psql -h $RDS -U dwh_user -d dwh \
  -f /etl/00_create_staging.sql
  
```

5.4 Étape 8.2 — Charger les CSV via \copy (côté client)

Pourquoi \copy et non COPY ?

RDS PostgreSQL ne permet pas COPY FROM '/chemin/fichier' car le fichier est sur le client (EC2), pas sur le serveur RDS. La commande \copy lit le fichier côté client psql et envoie les données via le flux réseau — c'est la solution correcte pour RDS.

```
docker run --rm -e PGPASSWORD='RGHgv5#Kp9mX2wQl' -v ~/data:/data postgres:16-alpine \
psql -h $RDS -U dwh_user -d dwh \
-c "\copy tmp_users FROM '/data/stg_users.csv' WITH (FORMAT csv, HEADER true, NULL '')" \
-c "\copy tmp_merchants FROM '/data/stg_merchants.csv' WITH (FORMAT csv, HEADER true, \
NULL '')" \
-c "\copy tmp_agencies FROM '/data/stg_agencies.csv' WITH (FORMAT csv, HEADER true, NULL \
'')" \
-c "\copy tmp_transactions FROM '/data/stg_transactions.csv' WITH (FORMAT csv, HEADER \
true, NULL '')"
```

5.5 Étape 8.3 — Peupler dimensions et fact_transactions

```
docker run --rm -e PGPASSWORD='RGHgv5#Kp9mX2wQl' -v ~/etl:/etl postgres:16-alpine \
psql -h $RDS -U dwh_user -d dwh \
-f /etl/02_populate_dim_date.sql \
-f /etl/03_populate_dim_user.sql \
-f /etl/04_populate_dim_merchant.sql \
-f /etl/05_populate_dim_agency.sql \
-f /etl/07_populate_fact_transactions.sql \
-f /etl/08_validation_checks.sql
```

5.6 Résultat attendu après ETL

Table	Lignes chargées	Type	Statut
dim_date	366	Dimension temps	✓
dim_user	10 000	Dimension utilisateur (SCD Type 2)	✓
dim_merchant	501	Dimension marchand (500 + ligne -1)	✓
dim_agency	101	Dimension agence (100 + ligne -1)	✓
dim_node	5	Dimension nœud serveur	✓
fact_transactions	100 000	Table centrale — FK orphelines : 0	✓

6. Déploiement Metabase & Connexion RDS

6.1 Étape 9 — Démarrer Metabase sur l'EC2

```
docker run -d \
  --name nafad_metabase \
  --restart unless-stopped \
  -p 3000:3000 \
  -e MB_DB_TYPE=h2 \
  -e MB_DB_FILE=/metabase-data/metabase.db \
  -v metabase_data:/metabase-data \
  metabase/metabase:v0.52.9

# Accès : http://<IP_EC2>:3000
```

6.2 Étape 10 — Connecter Metabase à RDS

Dans Metabase → Paramètres → Bases de données → Ajouter une base de données :

Champ	Valeur
Type	PostgreSQL
Nom	NAFAD DWH
Host	nafad-dwh-g4.cfqiicgoicgi.eu-west-3.rds.amazonaws.com
Port	5432
Database	dwh
Username	dwh_user
Password	RGHgV5#Kp9mX2wQI

6.3 Étape 11 — Elastic IP (IP fixe)

EC2 → Elastic IPs → Allocate Elastic IP Address → Associate → Sélectionner nafad-metabase-g4.

Pourquoi l'Elastic IP est indispensable

Sans Elastic IP, l'IP publique de l'EC2 change à chaque redémarrage. Cela invalide les règles SSH, les favoris navigateur, et les configurations Metabase qui referaient pointer vers l'ancienne IP. L'Elastic IP fixe l'adresse définitivement.

7. Commandes Utiles & Diagnostic

Commande	But
docker ps	Vérifier que nafad_metabase tourne

Commande	But
<code>docker start nafad_metabase</code>	Redémarrer Metabase si arrêté
<code>docker logs nafad_metabase</code>	Voir les logs Metabase
<code>docker logs nafad_metabase --tail 50</code>	Dernières 50 lignes de logs

```
# Vérifier le nombre de lignes dans fact_transactions
docker run --rm -e PGPASSWORD='RGHgv5#Kp9mX2wQl' postgres:16-alpine \
  psql -h nafad-dwh-g4.cfqiicgoicgi.eu-west-3.rds.amazonaws.com \
  -U dwh_user -d dwh \
  -c "SELECT COUNT(*) FROM fact_transactions;"
```

8. Problèmes Connus & Solutions

Problème	Cause	Solution
COPY from a file is not supported	RDS ne lit pas les fichiers locaux	Utiliser \copy (psql client) — voir étape 8.2
IP EC2 change au redémarrage	Pas d'Elastic IP assignée	Associer une Elastic IP — étape 11
Metabase ne redémarre pas après reboot	Option --restart absente	Ajouter --restart unless-stopped au docker run
SSH timeout	IP EC2 changée ou Security Group	Vérifier l'IP EC2 + mettre 0.0.0.0/0 dans SG si besoin

9. Estimation de Coût Mensuel

Service	Configuration	Coût/mois (USD)
RDS PostgreSQL 16	db.t3.micro · 20 GB	~25 \$ (Free tier 1 an)
EC2 Metabase	t3.medium · 20 GB EBS	~30 \$
Elastic IP	1 adresse associée	~0 \$ (gratuit si associée)
Data transfer	~10 Go/mois	~0,90 \$
TOTAL		~56 \$/mois (hors Free Tier)

10. Conclusion

L'architecture Early Stage de NAFAD PAY est opérationnelle sur AWS eu-west-3 (Paris) avec 100 000 transactions chargées dans le Data Warehouse RDS et le dashboard Metabase accessible via l'Elastic IP.

- › Infrastructure reproductible : les 11 étapes suffisent à tout recréer depuis zéro.
- › Sécurité réseau : RDS dans subnet privé, accessible uniquement depuis l'EC2 via Security Group.
- › ETL opérationnel : \copy contourne la limitation RDS (pas de COPY local).
- › Dashboard Metabase : v0.52.9 sur EC2, connecté à RDS, restart automatique.
- › Coût maîtrisé : ~56 \$/mois hors Free Tier AWS.